

Writing `vector` by Hand: Separating Capacity from Construction, and Exception Safety

CTC Practical C++ · Technical Report · 2026-09-15

This report documents a complete implementation of `my::vector<T>`. The problem comes from CTC's C++ phone screen (60 minutes, paired with `unique_ptr`), and requires rule of five, perfect forwarding, geometric growth, separation of capacity from construction, a relocation criterion, and exception safety.

The problem is not about algorithms. Everything it tests sits at the boundaries of the language's machinery: ownership, object lifetime, value categories, exception propagation paths. You can write application C++ for ten years and never touch any of it; people who write libraries work there every day.

1 The core mechanism: capacity separated from construction

The whole problem rests on one sentence: **`vector` must be able to operate on memory that is allocated but not constructed.**

The reason is that `capacity` may exceed `size`. Those extra slots are real memory, but no objects may live in them — otherwise `reserve(1000)` would conjure a thousand objects out of nothing, and the whole thing would fail to compile the moment `T` has no default constructor.

So `new T[n]` is ruled out from the start: it welds allocation and construction together. Prying them apart takes four operations, which pair up two by two.

Operation	What it does	What it does not do
<code>::operator new(n)</code>	Allocates <code>n</code> bytes of raw memory	Calls no constructor
<code>new (p) T(args)</code>	Constructs an object at address <code>p</code>	Allocates nothing
<code>p->~T()</code>	Destroys a single object	Frees no memory
<code>::operator delete(p)</code>	Frees raw memory	Calls no destructor

THE CONSEQUENCE THAT MATTERS

RAII is deliberately broken here. The pairing that `new/delete` expressions do for you — allocate+construct, destroy+free — now falls entirely on you. **Every object built with placement new owes exactly one explicit destructor call.** That pairing is the premise behind every exception-safety argument that follows.

There is a zero-cost test for whether you actually did this: give `T` no default constructor.

```
struct NoDefault { int v; explicit NoDefault(int a) : v(a) {} };

my::vector<NoDefault> v;
v.reserve(100);          // an implementation using new T[n] fails to compile here
```

2 `reserve` is the only place resources move

Five functions need growth: both `push_back` overloads, `emplace_back`, `resize`, and `reserve` itself.

WHY NOEXCEPT IS NOT DECORATION

Leave `noexcept` off your move constructor and the enclosing container's growth degrades from an $O(1)$ pointer steal to an $O(n)$ deep copy. That is a behavioral difference, not a style preference.

A distinction that is easy to blur

`move_if_noexcept` belongs to **relocation during growth** only. Inside `push_back(T&&)` the right call is an unconditional `std::move`:

```
void push_back(T&& val) {
    // ...
    new (ptr_ + size_) T(std::move(val)); // not move_if_noexcept
}
```

The difference is whose responsibility is at stake. Elements relocated during growth are the container's own; wrecking them is the container's fault. The argument to `push_back(T&&)` was handed over by the caller as an rvalue — ownership has already transferred, and taking it unconditionally is the correct semantics.

4 Exception safety

reserve: the strong guarantee

Only the construction loop needs protection. `::operator new` sits outside the `try` — when it throws, no memory was obtained, and `ptr_`, `size_`, `capacity_` are all untouched. There is nothing to roll back.

```
T* mem = static_cast<T*>(::operator new(sizeof(T) * val));
std::size_t i = 0;
try {
    for (; i < size_; i++) {
        new (mem + i) T(std::move_if_noexcept(ptr_[i]));
    }
}
catch (...) {
    for (std::size_t j = 0; j < i; j++) { mem[j].~T(); }
    ::operator delete(mem);
    throw;
}
```

Three details:

- **`i` is declared outside the `try`.** The handler needs to know how many objects were constructed; declared inside the `for`, that boundary is unreachable.
- **Destroy one by one, then free.** `::operator delete` runs no destructors; drop that loop and every heap resource held by the constructed objects leaks.
- **`catch (...)` paired with `throw`;** `T`'s constructor may throw anything at all, not necessarily something derived from `std::exception`. Writing `throw e`; throws a sliced copy and loses the dynamic type; a bare `throw`; rethrows the original object.

`push_back`: strong, with one loose end

`push_back` is `reserve` plus one placement new. If that construction throws, the object was never created and `++size_` never ran — `size_` still names the pre-call element count, the failed slot is not counted, and the state is consistent.

But a strict strong guarantee demands "no effects," and growth has already happened: `capacity_` changed and every old iterator is invalid.

THE TRADE-OFF

The standard library **constructs the new element into the new buffer first, then relocates the old ones**. If the new element's constructor throws, the entire new buffer is discarded and the vector was never touched — full no-effects. The cost is a more convoluted ordering.

This implementation grows first and constructs second, achieving: **after a throw, `size()` and the existing elements are unchanged and nothing leaks, but capacity may have grown and iterators are invalidated**. Say that sentence out loud in the interview rather than claiming full no-effects.

Where try/catch is not needed

```
try {
    new (ptr_ + size_) T(val);
    size_++;
}
catch (...) { throw; }    // no-op; delete it
```

The handler performs no cleanup and rethrows unchanged — remove the block and the behavior is identical. The test for exception safety is "did I create state that needs rolling back," not "can this throw."

Exceptions from destructors

Destructors are `noexcept` by default; throwing calls `std::terminate`. Even with an explicit `noexcept(false)`, throwing during stack unwinding terminates anyway — and `reserve` destroying old elements can sit precisely on a cleanup path.

Conclusion: **this is not a path you can handle, so it does not go inside a try block**. The standard library's contract simply requires that `T`'s destructor not throw, placing the responsibility on `T`'s author.

5 Four traps in the rule of five

Move operations: steal, don't ferry

A common wrong implementation routes through `reserve` and moves element by element. The result is correct but an order of magnitude more expensive — `n` move constructions plus `n` old objects awaiting destruction — and because it allocates and can throw, it cannot be marked `noexcept`.

The right version is an $O(1)$ steal:

```
vector(vector&& other) noexcept {
    ptr_ = other.ptr_;
    size_ = other.size_;
    capacity_ = other.capacity_;
    other.ptr_ = nullptr;
    other.size_ = 0;           // all three must be cleared
    other.capacity_ = 0;
}
```

WHY ALL THREE MEMBERS

Clearing `ptr_` alone is not enough. `other`'s destructor loops for `(i = 0; i < size_; ++i)` `ptr_[i].~T();` — with `size_` still 5 and `ptr_` null, the first iteration dereferences a null pointer.

Even if a null check in the destructor prevents the crash, `other.size()` still reports 5 and `empty()` still returns `false`. The object is lying about itself.

A moved-from object is **valid but unspecified**: it can be destroyed, assigned to, cleared — you simply should not read its former value. It is neither "invalid" nor "unusable."

Self-assignment checks belong to assignment, not construction

`if (this != &other)` is load-bearing in an **assignment** operator and dead code in a **constructor** — an object under construction cannot possibly be `other`. Worse, if that impossible condition were false, all three members would be left uninitialized.

In move assignment it must be there:

```
vector& operator=(vector&& other) noexcept {
    if (this != &other) {
        clear();                // destroy own elements
        ::operator delete(ptr_); // free own memory
        ptr_ = other.ptr_; /* ... steal ... */
        other.ptr_ = nullptr; /* ... clear ... */
    }
    return *this;
}
```

Without the check, `v = std::move(v)` destroys its own resources first and then reads a pointer out of freed memory.

A COMMON MISCONCEPTION

"`v = std::move(v)` is UB, so it doesn't matter" — not so. That statement is legal code; the compiler only emits `-Wself-move`. The UB is manufactured by *your implementation*, not mandated by the language. The standard requires valid-but-unspecified for standard library types; for a class you wrote, the behavior is whatever you make it.

Copy-and-swap: the other route

Copy assignment has a much shorter form:

```
vector& operator=(const vector& other) {
    vector tmp(other);
    swap(tmp);
    return *this;
}
```

It reuses the copy constructor; `swap` exchanges three members and `tmp`'s destructor carries away the old data. Self-assignment safety and the strong guarantee both come for free. The cost is one extra allocation. Either form passes; the choice depends on the context.

6 Const correctness: which functions need two versions

The criterion is not "does this function modify the object" but **does the return value expose the internals**.

Function	Returns	Versions needed	Why
<code>size()</code> <code>capacity()</code> <code>empty()</code>	a value	one, const	the caller gets a copy and cannot write back
<code>operator[]</code> <code>at()</code>	a reference	two	references can write into the container
<code>begin()</code> <code>end()</code>	a pointer	two	pointers can write into the container

A non-const object may call a const member function, so value-returning accessors need only the const version. Anything returning a reference or pointer needs a pair — `const T&` / `const T*` on const objects, `T&` / `T*` otherwise.

Incidentally, range-for requires only `!=`, prefix `++`, and unary `*` from its iterator, all built in for raw pointers. `T*` is entirely sufficient; wrapping an `iterator` class buys type safety, not syntax.

7 A taxonomy of the mistakes

Setting aside the typos a compiler catches outright, every substantive error made during this implementation collapses into four categories:

#	Category	Typical form
1	Stealing a resource without clearing the source	Move constructor clears <code>ptr_</code> but not <code>size_ / capacity_</code> ; double free
2	Freeing memory without destroying objects	<code>catch</code> calls <code>::operator delete(mem)</code> directly; every constructed object's resources leak
3	No rollback on the exception path	<code>break</code> inside <code>catch</code> swallows the exception; <code>resize</code> 's grow branch left unguarded
4	Missing <code>const</code> / <code>noexcept</code>	<code>begin()</code> only const but returning <code>T*</code> ; move constructor not <code>noexcept</code>

Two further traps appear only inside templates and are worth recording separately:

- **A member function of a class template is not instantiated until it is called.** A `return ptr;` with a completely wrong type can hide inside an array specialization's move assignment and survive an entire test suite, as long as nothing calls it. Every member needs at least one call in your tests — or force it with `template class my::vector<int>;`.
- **Pointer arithmetic scales implicitly.** In `mem + offset` the compiler has already multiplied by `sizeof(T)`; writing `offset += sizeof(T)` multiplies a second time.

8 Verification

The final implementation passes 1121 checks, clean under ASan/UBSan, with zero warnings under `-Wall -Wextra`. Coverage:

- Empty vector; `push_back` lvalue/rvalue routing; `emplace_back` in-place construction and perfect forwarding (lvalue vs rvalue)
- `operator[]` / `at()` in const and non-const form; `at()` throwing `std::out_of_range`
- `reserve` constructing nothing; `resize` in both directions; `clear` preserving capacity
- Geometric growth: 1000 `push_back` calls trigger only 11 reallocations
- Full rule of five, including self-assignment and self-move-assignment, and a reusable moved-from source
- Types with no default constructor; move-only types
- Iterator invalidation across reallocation
- `move_if_noexcept` routing verified with two separate probe types
- `push_back` rollback: a copy constructor throwing mid-relocation leaves size and elements unchanged with zero net change in live objects
- 200×5 rounds of push/pop churn with no leaks; full `std::string` workflow

9 Notes for the interview

Writing 180 lines in sixty minutes — alongside `unique_ptr` — is not realistic. By weight:

1. **`reserve` and `emplace_back` must be written.** Capacity/construction separation, perfect forwarding, and exception rollback all live in these two functions.
2. **The rule of five must be written**, the move operations above all — steal rather than ferry, clear all three members, check self-assignment, mark `noexcept`.
3. `push_back` is two lines of forwarding; `pop_back` and the accessors are a few lines each.
4. Anything left unwritten, such as `resize`, can simply be described out loud.

Likely follow-ups, worth having ready:

- Why can't you use `new T[n]`?
- When does relocation move and when does it copy, and what is the criterion?
- Which exception-safety guarantee does `push_back` provide, and does your implementation actually reach it?
- Why does `vector` have two `push_back` overloads but only one `emplace_back`? (In `push_back(T&&)`, `T` is a class template parameter, already fixed, so this is an rvalue reference. In `emplace_back(Args&&...)`, `Args` is a function template parameter deduced at the call site, making it a forwarding reference.)
- Why a growth factor of 2, and why do some implementations use 1.5? (At 2×, the sum of all previously freed blocks always falls just short of the next request. At 1.5×, accumulated free blocks eventually cover a new request — provided the allocator coalesces adjacent free blocks.)

Footnote: `::operator new` only guarantees `__STDCPP_DEFAULT_NEW_ALIGNMENT__` (commonly 16 bytes). An over-aligned `T` — `alignas(64)`, say — requires the C++17 `std::align_val_t` overload. Out of scope for this problem, but cache-line-aligned types are common in low-latency work.